

6주차_A

사람 유예원

▼ B의 4가지 질문에 대한 답

1. `errors` 전달 경로 — 이미 반영됨. `main.py` 가 `result.get("errors")` 를 직접 보고 `module_status` 를 ok/partial로 나눈다. 레거시 스키마를 안 쓰니 "risk_score 실패 시 0.0으로 강제" 문제 자체가 없다 — 실패하면 `risk_score` 가 그냥 `None` 으로 나오고 `status="partial"` 이 같이 뜨니까 "0=안전"으로 오독될 일이 없다.
2. manifest 파싱 실패 정책 — "빈 배열로 계속 진행"이 아니라 그냥 원본 `analyze_static()` 의 정책(부분 실패 시 예외 없이 `None` + `errors` 에 기록)을 그대로 존중한다. 어댑터를 안 쓰니 이 질문은 사실상 해소됨.
3. `min_sdk` / `target_sdk` 의 0 처리 — 레거시 스키마 검증은 안 하니 null 허용 여부 이슈가 없다. 원본 값(0 또는 정상값) 그대로 통과시키고, "0=파싱실패"인지 구분해서 보여주는 건 7~8주차 대시보드 몫으로 미룬다.
4. `certificate` / `code_analysis` / `strings` / `third_party_sdks` 자리 — 이미 있음. 8필드 그대로 통과시키니 버려지는 필드가 없다.

D에게 제안한 "strings.urls/ip_addresses ↔ 네트워크 suspicious.domains/ips 교차 검증"은 좋은 아이디어라 스펙 문서에 D 몫으로 남겨두겠다.

▼ 브랜치 전략

main 직접 push 금지, PR은 열어두되 A/C/D가 6주차 나머지 끝난 뒤에 한꺼번에 merge. 그럼 지금 내 `main.py` 도 실제로는 각 브랜치 파일을 로컬에서 임시로 모아 테스트하는 식으로만 검증 가능하고(내가 앞서 검증할 때 한 방식 그대로), origin에는 아직 못 올린다 — 이것도 다른 역할들과 동일한 처리라 문제 없다.

C(은아)의 "이제 `output` 폴더에 저장"에 맞춰 `main.py` 의 `run_scenario()` 호출에 `report_dir` 인자를 안 넣고 있던 걸 고쳐두겠다.

명령 5개 실행함, 3개 파일 수정됨

명령 5개 실행함, 3개 파일 수정됨

두 파일 다 갱신해서 프로젝트 폴더에 반영했다. 스펙 문서에 B와의 방향 차이 정리, 4개 질의 답변, 브랜치/PR 운영 방침, C의 `output` 폴더 변경사항을 추가했고, `main.py` 는 `report_dir="output"` 반영 + B 어댑터를 안 쓰는 이유를 주석으로 남겼다.

핵심은 이거다: schemas/ 폴더 비교는 이제 안 해도 된다는 판단이 맞고, `main.py` 는 원래부터 8필드 원본을 그대로 쓰도록 짜여 있어서 코드는 안 바뀐다. 다만 이 결정을 B가 작업 시작 전에 몰랐던 게 문제였으니, 이 스펙 문서를 지금 바로 팀에 공유해야 한다 — 그래야 C/D도 같은 혼선 없이 진행할 수 있다.

▼ [역할1] 6주차 통합 인터페이스 스펙 공유 + `main.py` 골격 완료

Day1 스펙 초안을 문서로만 만들어두고 실제 공유를 안 해서 B(은서)가 못 보고 독자적으로 진행한 걸 뒤늦게 확인했습니다. 죄송합니다 — 지금 바로 공유합니다.

문서: 6주차_Day1_통합인터페이스_스펙.md (첨부)

핵심만 요약:

- 정적/동적/네트워크 각 모듈의 `schema.py`를 그대로 기준으로 삼습니다. schemas/ 폴더의 1주차 초안 스키마는 이번 통합에서 안 씁니다(레거시).
- `main.py`는 각 모듈 결과를 변환 없이 그대로 통과시킵니다 — 어댑터 함수 안 씀. static은 8필드, dynamic은 `ScenarioResult` 그대로, network는 D의 `build_network_report()` 그대로.
- 에러 정책: 모듈별 fatal exception은 `module_status(ok/partial/failed/timeout)`로 흡수, 한 모듈이 죽어도 나머지는 계속 진행 + `report.json`은 항상 나옴.
- 실행 순서: 정적(먼저, 독립) → 동적+네트워크(동시 실행, `capture_during_scenario`로 동기화) → 위험도 스코어링(D).

main.py 골격 작성 + 로컬 검증(성공/static실패/dynamic크래시/timeout 4개 시나리오) 끝났습니다. 브랜치에 곧 push하고 PR 열겠습니다 — B/C/D는 그때부터 각자 코드가 이 골격에 꽂히는지 확인 가능합니다.

각자 답변 필요한 항목은 개인 DM으로 따로 보낼게요 (B/C/D 순서).